

Katmanlı Analiz ve Test Altyapısı: Yazılım Sistemlerini Anlamak İçin İki Çerçeve

Ömer Faruk Yavuz

July 2026

Giriş

Büyük dil modelleri yazılım geliştirme pratiğine girdikçe, “bir projeyi yapay zekâya nasıl okuturuz” sorusu giderek daha kritik bir mühendislik problemi hâline gelmiştir. Bir kod tabanını Claude gibi bir modele analiz ettirmek, ilk bakışta basit bir dosya besleme işi gibi görünse de, gerçekte bir bilgi mimarisi ve karar tasarımı meselesidir. Dosyaların hangi sırayla, hangi amaçla ve hangi sorular eşliğinde okutulduğu, ortaya çıkan analizin kalitesini doğrudan belirler. Rastgele ya da mekanik bir okuma düzeni, modelin yalnızca dosya taraması yapmasına yol açar; oysa asıl hedef, modelin sistem düzeyinde düşünmesini sağlamaktır.

Bu belge iki ana bölümden oluşmaktadır. Birinci bölüm, bir projeyi Claude’a okutmanın katmanlı ve karar odaklı çerçevesini ortaya koyar: önce mimari haritanın çıkarılması, ardından risk ve kırılabilirlik analizinin yapılması, sonra aksiyon ve önceliklendirme kararlarının üretilmesi, en sonunda ise kodun arkasındaki tarihsel, organizasyonel ve operasyonel bağlamın hesaba katılması. Bu dört katman birlikte, tek seferlik bir taramanın veremeyeceği derinlikte bir sistem analizi üretir. Ayrıca bu çerçevenin doğal bir uzantısı olarak, repodan güvenilir bir `CLAUDE.md` üretme mantığı da ele alınır.

İkinci bölüm ise test altyapısının kavramsal sözlüğüne odaklanır. *Test rig* kavramı, sektörde sıkça kullanılan ama sınırları çoğu zaman belirsiz kalan bir terimdir. Bu bölüm, test rig’i oluşturan alt bileşenleri — test verisi katmanından test doubles yapılarına, ortam kurulumundan doğrulama ve temizlik mekanizmalarına kadar — sistematik bir taksonomi hâlinde sunar. Böylece hem günlük mühendislik iletişimde hem de teknik dokümantasyonda doğru terminolojiyle konuşabilmek için sağlam bir referans oluşturur.

Her iki bölümün ortak paydası şudur: karmaşık bir teknik olguyu tek bir etikete indirgemek yerine, onu bileşenlerine ayırmak, katmanlandırmak ve amaç odaklı bir düzene oturtmak. Bu yaklaşım, hem yapay zekâ destekli kod analizinde hem de test altyapısı tasarımında aynı zihinsel disiplinin ürünüdür.

1 Claude’a Proje Okutma Yaklaşımı: Katmanlı, Sistematik ve Karar Odaklı Çerçeve

Bir projeyi Claude’a okutmanın doğru yolu, dosyaları rastgele ya da yalnızca boyutlarına göre sıraya dizmek değildir. Çünkü küçük dosya bazen en az önemli dosyadır; buna karşılık kısa bir routing dosyası, bir dependency injection konfigürasyonu, bir auth guard, bir env ayarı ya da root reducer dosyası tüm sistemin davranışını belirleyen kritik bir düğüm olabilir. Bu yüzden temel ilke şudur: **dosya boyutuna göre değil, bilgi yoğunluğuna ve karar değerine göre okutmak gerekir.**

Amaç, önce sistemi genel hatlarıyla anlamak, sonra davranışı ve veri akışını görmek, daha sonra riskleri ve kırılganlıkları ayıklamak, en sonunda ise hangi işin hangi sırayla ele alınacağını belirlemektir. Bunun da ötesinde, yalnızca kodu değil, kodun neden o hâle geldiğini de anlamak gerekir. Bu nedenle en güçlü yaklaşım tek tur değil, katmanlı bir analiz yapısıdır.

1.1 Merkez İlke

Bu yaklaşımın kalbi şu cümlede toplanabilir:

Önce sistemi anlamasını sağla, sonra detaylara indir, sonra riskleri gör, sonra da neyin hangi sırayla yapılacağını zorla.

Başka bir ifadeyle:

İlk tur haritayı çıkarır. İkinci tur mayın tarlasını gösterir. Üçüncü tur hangi yolu açacağını söyler. Dördüncü tur ise o yolun neden kapandığını açıklar.

2 Neden “En Küçük Dosyadan Başla” Yaklaşımı Yetersizdir

Bazı insanlar bir projeyi analiz ederken “küçük dosyadan başlayalım” gibi mekanik bir kural önerebilir. Bu yaklaşım yüzeyde pratik görünse de aslında eksiktir. Çünkü küçük dosya her zaman düşük önem anlamına gelmez. Tersine, proje giriş noktası, routing konfigürasyonu, dependency injection yapısı, environment dosyası, ana state kaynağı ya da sistemin bootstrapping akışı çoğu zaman küçük ama kritik dosyalarda bulunur.

Bu nedenle okuma sırası teknik olarak şöyle kurulmalıdır:

1. önce sistemin ne yaptığı ve nasıl ayağa kalktığı anlaşılır,
2. sonra sistemin hangi yapılardan oluştuğu görülür,
3. sonra gerçek veri ve kontrol akışı izlenir,
4. sonra risk ve değişim maliyeti çıkarılır,

5. en sonunda aksiyon ve önceliklendirme yapılır.

Bu yapı olmadan Claude dosya okur ama sistem düşünemez. Oysa hedef, dosya bazlı bir tarama değil, **sistem bazlı analiz** üretmektir.

3 Birinci Katman: Sistemi Tanıma ve Mimari Haritayı Çıkarma

Birinci katmanın amacı sistemi tanımadır. Burada henüz asıl soru “neresi bela” değildir. Asıl soru şudur:

Bu proje nasıl çalışıyor ve hangi ana yapılardan oluşuyor?

Bu aşamada Claude’a önce şu alanlar okutulmalıdır:

3.1 Projenin Amacı ve Sınırları

Bir projenin teknik yapısını anlamadan önce onun hangi problemi çözdüğünü, neyi kapsadığını ve neyi kapsamadığını görmek gerekir. Burada şu sorular sorulmalıdır:

- Bu proje tam olarak ne yapıyor?
- Ne yapmıyor?
- Hangi kullanıcı tiplerine hizmet ediyor?
- Ana kullanım senaryoları neler?
- Ürünün merkezindeki problem nedir?

Bu sorular olmadan teknik analiz bağlamsız kahr.

3.2 Kök Dosyalar ve Konfigürasyon

Ardından README, docs, setup notları, package.json, pyproject.toml, pom.xml, go.mod, nx.json, angular.json, tsconfig, lint ve format ayarları, Dockerfile, docker-compose, helm ve CI dosyaları incelenmelidir. Bu seviyede amaç şunları çıkarmaktır:

- teknoloji yığını nedir,
- build, test ve lint akışı nasıl kurulmuştur,
- monorepo yapısı var mıdır,
- deployment ile ilgili temel ipuçları nelerdir.

3.3 Çalıştırma ve Ortam Mantığı

Bir proje kağıt üzerinde iyi görünebilir ama local, stage ve prod davranışı birbirinden çok farklı olabilir. Bu nedenle environment kullanımı ayrıca incelenmelidir. Örneğin:

- env dosyaları nasıl organize edilmiştir,
- feature flag yapısı var mıdır,
- config değerleri build time mı runtime mı gelir,
- ortamlar arası farklar nasıl yönetilir.

3.4 Entry Point'ler

Sistem ilk nereden ayağa kalkıyor sorusu çok kritiktir. Frontend'de bootstrap dosyaları, backend'de main, app ya da server giriş noktaları, varsa worker, cron ya da queue consumer başlangıçları birlikte okunmalıdır. Çünkü dependency injection, modül yükleme sırası ve sistemin ilk ayağa kalkış mantığı burada görünür.

3.5 Route, Module ve Feature Haritası

Bir sonraki adım route ağacını, modülleri, lazy-loaded alanları ve feature klasörlerini çıkarmaktır. Buradaki amaç, sistemin hangi parçalara bölündüğünü anlamaktır. Claude bu seviyede şu tür sorulara cevap vermelidir:

- Sistem hangi feature'lara bölünmüş?
- Kullanıcı hangi ana akışlardan geçiyor?
- Bounded context benzeri ayrımlar var mı?
- Hangi modüller birbirine daha sıkı bağlı?

3.6 Core, Shared ve Common Katmanları

Ardından ortak altyapı okunmalıdır. Bu bölümde reusable component'ler, ortak servisler, helper ve util katmanları, interceptors, guards, middleware, auth ve error handling yapıları analiz edilir. Çünkü sistemin tekrar eden davranışları ve ekip içi gerçek convention'ları çoğu zaman burada saklıdır.

3.7 Domain Modeli ve Temel Kavramlar

Bu bölüm çoğu kişinin atladığı ama aslında en kritik alanlardan biridir. Çünkü sistem yalnızca dosyalardan oluşmaz; bir kavram dünyasından oluşur. Bu yüzden Claude'un şu soruları da cevaplaması gerekir:

- Sistemin ana varlıkları nelerdir?

- Bu varlıklar birbirine nasıl bağlanır?
- Aynı şeyi anlatan birden fazla model var mı?
- İsimlendirme tutarlı mı?
- İş dili ile kod dili birbiriyle örtüşüyor mu?

3.8 Veri Modeli, API ve State Akışı

Bu aşamada DB şeması, entity'ler, repository yapıları, migration dosyaları, seed data, transaction mantığı, cache kullanımı, request-response modelleri, DTO yapıları, validation, serialization, backend kontratları ve frontend state akışı birlikte görülmelidir.

Özellikle modern frontend projelerinde şu alanlar ayrıca incelenmelidir:

- local component state,
- global store,
- NgRx action, reducer, effect ve selector akışları,
- optimistic update var mı,
- derived state var mı,
- duplicated state var mı,
- single source of truth nerede.

3.9 Ana İş Akışları

Daha sonra login, search, upload, payment, appointment oluşturma, recording başlatma, transcript alma, summary üretme gibi iş akışları feature bazında açılmalıdır. Her akış için şunlar çıkarılmalıdır:

- giriş noktası,
- kritik adımlar,
- success ve failure yolları,
- state değişimleri,
- API çağrıları,
- kullanıcıya görünen sonuç.

3.10 Birinci Katmanın Sonucu

Birinci turun çıktısı şunlar olmalıdır:

- proje mimarisi özeti,
- en kritik dosyalar,
- ana iş akışlarının kısa özeti,
- veri ve kontrol akışı haritası,
- yeni bir geliştirici için önerilen okuma sırası.

Bu katman sana **haritayı** verir. Ama henüz mayın tarlasını vermez.

4 İkinci Katman: Risk, Kırılabilirlik ve Değişim Maliyeti Analizi

İkinci katmanın amacı sistemi anlatmak değil, sistemi zorlayan alanları bulmaktır. Buradaki ana soru şudur:

Bu projede nereler kırılabilir, nereler pahalı, nereler küçük değişiklikte zincirleme etki yaratıyor?

Bu turda özellikle aşağıdaki analiz eksenleri eklenmelidir.

4.1 Değişim Etkisi Analizi

Bazı dosyalar büyük değildir ama sistemi birbirine bağlayan yapıştırıcı işlevi görür. Bu yüzden Claude'a şu sorular sordurulmalıdır:

- Küçük bir değişiklik yapıldığında en çok zincirleme etki yaratan alanlar nereler?
- Hangi dosyalar çok fazla yere bağlı?
- Hangi modüller low cohesion ve high coupling gösteriyor?
- Hangi değişiklikler regression riski taşıyor?
- Hangi alanlara dokunmak pahalı?

4.2 Kırılğanlık Analizi

Bu bölümde özellikle hata anındaki davranış incelenmelidir:

- Hangi akışlar hata durumunda kötü degrade oluyor?
- Nerede fallback yok?
- Nerede null, undefined ya da state timing problemi çıkabilir?
- Nerede race condition veya async ordering problemi olabilir?
- Nerede hata sessizce yutuluyor?

Özellikle frontend, NgRx ve async backend kombinasyonlarında bu alan altın değerindedir.

4.3 Teknik Borç Sınıflandırması

Teknik borcu tek torbaya atmak yetersizdir. Claude'un teknik borcu sınıflandırması gerekir. Örneğin:

- yapı borcu,
- test borcu,
- naming ve model borcu,
- duplication borcu,
- state borcu,
- legacy borcu,
- dependency borcu,
- environment ve config borcu.

Buradaki amaç, “burada teknik borç var” demek değil, **ne tür borç var** demektir.

4.4 Gerçek Karmaşıklık Analizi

Bazı kodlar uzun ama düzdür. Bazıları ise kısadır ama zehirlidir. Bu yüzden şu sorular da sorulmalıdır:

- En karmaşık görünen dosyalar hangileri?
- En yanıltıcı biçimde basit görünen ama aslında riskli dosyalar hangileri?
- Branching'in en yoğun olduğu yer neresi?
- Business rule yoğunluğu hangi katmanda toplanmış?
- Hidden complexity nerede?

4.5 Test Kör Noktaları

Bu turda testlerin varlığı değil, testlerin neyi kaçırdığı önemlidir:

- En kritik ama en az test edilen akışlar hangileri?
- Testler hangi riskleri hiç yakalamıyor?
- Mock testler hangi gerçeği gizliyor?
- Nerede integration test eksikliği tehlikeli?
- Nerede contract test gerekir?
- Hangi akışlar E2E olmadan güven vermez?

4.6 State ve Veri Tutarlılığı Riski

Bu başlık özellikle önemlidir:

- Aynı veri birden çok yerde tutuluyor mu?
- Derived state ile source state çakışıyor mu?
- Frontend state ile backend response arasında uyumsuzluk olabilir mi?
- DB migration, schema ve seed tarafında tutarsızlık riski var mı?
- Hangi alanlarda stale data problemi doğabilir?

4.7 Async ve Entegrasyon Riski

Dış servisler, retry mekanizmaları, websocket, SignalR, polling ve event akışları sistemin en kırılgan alanlarından biri olabilir. Şu sorular özellikle önemlidir:

- Yarış durumu çıkabilecek alanlar nereler?
- Retry, idempotency ya da ordering problemi hangi akışlarda doğabilir?
- Dış servis hatalarında sistem nasıl davranıyor?
- Timeout, circuit breaker ya da fallback koruması eksik mi?

4.8 İkinci Katmanın Sonucu

Bu turun sonunda Claude şunları çıkarabilmelidir:

- en kırılgan 10 bölge,
- en riskli 10 dosya veya modül,
- en büyük regression riskleri,

- en önemli test boşlukları,
- hemen yapılması gereken refactorlar,
- sonraya bırakılabilecek refactorlar,
- yeni feature geliştirirken dikkat edilmesi gereken alanlar.

Bu katman sana **mayın tarlasını** verir.

5 Üçüncü Katman: Aksiyon, Önceliklendirme ve Uygulama Sırası

Üçüncü turda artık soru “ne var” ya da “neresi riskli” değildir. Asıl soru şudur:

Peki şimdi ne yapacağız?

Bu katmanın amacı karar vermektir. Teknik analiz yapılmış, riskler görülmüştür. Şimdi bunların hangi sırayla ele alınacağı belirlenmelidir.

5.1 Önceliklendirme

Bu seviyede Claude şu tür karar sorularına zorlanmalıdır:

- İlk 5 öncelik ne olmalı?
- Hangi problemler kök neden seviyesinde?
- Hangileri çözülmürse başka problemler de azalır?
- Hangi sorunlar can sıkıcı ama bekleyebilir?

5.2 ROI ve Uygulanabilirlik

Her refactor aynı değeri üretmez. Bu nedenle şunlar sorulmalıdır:

- En yüksek etki ve en düşük maliyetli iyileştirmeler hangileri?
- Hangi refactor geliştirici hızını hemen artırır?
- Hangi düzeltme bug riskini hızlıca düşürür?
- Hangi işler küçük ve güvenli parçalara bölünebilir?
- Hangi alanlarda önce test veya observability eklenmeden değişiklik yapmak risklidir?

5.3 Refactor Stratejisi

Burada refactor'lar üçe ayrılmalıdır:

- hemen yapılması gerekenler,
- feature geliştirme sırasında opportunistic yapılabilecekler,
- ayrı epic olarak planlanması gerekenler.

Ayrıca hangi alanlara şimdilik dokunmamak gerektiği de açıkça yazılmalıdır. Çünkü bazen iyi mühendislik, doğru refactor'u yapmak değil, yanlış zamanda refactor yapmamaktır.

5.4 Yol Haritası

Bu katmanda sonuç sadece soyut tavsiye olmamalıdır. Zaman ölçeği verilmelidir:

- 1 hafta içinde yapılacak quick win'ler,
- 1 ay içinde yapılacak orta ölçekli iyileştirmeler,
- 1 çeyrek içinde yapılacak stratejik mimari adımlar.

5.5 Karar Filtresi

Burada çok önemli bir filtre gerekir. Claude'a her öneri için şunlar sordurulmalıdır:

- Bu öneri gerçekten kullanıcıya, takıma ya da güvenilirliğe değer katıyor mu?
- Bu yalnızca kodu daha temiz mi gösteriyor, yoksa gerçek risk azaltıyor mu?
- Bu değişiklik yapılmazsa ne olur?
- Bu değişiklik yapılırsa ne kazanılır?
- Bu işi şimdi yapmak mı mantıklı, yoksa sonraki büyük feature ile birlikte mi?

5.6 Üçüncü Katmanın Sonucu

Bu turun sonunda Claude şu çıktıları verebilmelidir:

- önceliklendirilmiş teknik backlog,
- ilk 3 quick win,
- ilk 3 stratejik yatırım,

- dokunulmaması gereken alanlar,
- güvenli refactor sırası,
- 30 günlük önerilen uygulama planı.

Bu katman sana **hangi yolu açman gerektiğini** söyler.

6 Dördüncü Katman: Tarih, Baskı ve Gerçek Dünya Bağlamı

İlk üç katman kodu ve yapıyı okur. Dördüncü katman ise şunu anlamaya çalışır:

Bu sistem neden böyle şekillendi ve gerçekte nerede acı veriyor?

Bu katman olmadan Claude bazen teknik olarak çok etkileyici ama iş açısından yanlış öncelikler verebilir.

6.1 Mimari Karar Geçmişi

Bazı yapılar kötü yazıldığı için değil, zorunlu olduğu için vardır. Bu nedenle şu sorular sorulmalıdır:

- Bu yapı neden böyle seçildi?
- Geçmişte hangi alternatifler düşünülmüştü?
- Hangi karar bilinçli, hangisi geçici?
- Hangi kısıtlar bu çözümü zorladı?
- Hangi parçalar eski sistemden taşındı?

Buradaki amaç, **kötü kod mu, zorunlu kod mu** ayrımını yapmaktır.

6.2 Bug ve Incident Geçmişi

Gerçek kırılabilirlik çoğu zaman sadece geçmiş arızalarda görünür. Bu nedenle şu alanlar önemlidir:

- Geçmişte en çok hangi alanlar patladı?
- Hangi akışlar tekrar tekrar sorun çıkardı?
- Prod incident'lar hangi modüllerde yoğunlaştı?
- Hangi hatalar kullanıcıya gerçek zarar verdi?
- Hangi problemler sessiz veri bozulmasına yol açtı?

6.3 PR ve Commit Paterni

Kodun stres altında kalan bölgesini anlamamanın en iyi yollarından biri churn’e bakmaktır:

- En sık hangi dosyalar değişiyor?
- Hangi modüller sürekli hotfix alıyor?
- Hangi alanlarda küçük iş bile büyük diff yaratıyor?
- Hangi kod parçaları sürekli elden geçiriliyor ama düzelmiyor?

6.4 Takımın Yaşadığı Gerçek Acılar

Kod düzgün görünebilir ama ekip o alanda sürekli zorlanıyorsa orası zaten problemdir. Bu yüzden şu tür sorular da gereklidir:

- Yeni gelen geliştirici en çok nerede zorlanır?
- Review’larda en çok hangi alanlar tartışma çıkarır?
- Hangi modüller “kimse dokunmak istemiyor” hissi verir?
- Hangi alanlarda ownership belirsizdir?
- Hangi yapılar takım hızını sürekli düşürür?

6.5 Business Kritikliği

Teknik olarak çirkin bir alan, iş açısından dokunulamaz olabilir. Bu yüzden şu sorular şarttır:

- Hangi akışlar para, kullanıcı güveni veya operasyon için kritik?
- Hangi hata tolere edilebilir, hangisi edilemez?
- En önemli kullanıcı akışları hangileri?
- Support yükünü en çok hangi feature üretiyor?
- İş açısından “çalışması şart” olan parçalar hangileri?

6.6 Operasyonel Gerçeklik

Bazı sorunlar yalnızca canlı ortamda görünür. Bu nedenle observability ve prod davranışı ayrıca incelenmelidir:

- Prod’da en zor debug edilen akışlar hangileri?
- Loglarda görünmeyen ama sahada hissedilen sorunlar neler?

- Monitoring ve alerting hangi alanlarda yetersiz?
- Retry, timeout, rate limit ve fallback gibi korumalar nerede eksik?
- Hangi akışlar sadece canlı ortamda sorun veriyor?

6.7 Geçici Çözümler ve Yarım Kalan Geçişler

Dördüncü turda ayrıca şu alanlar da aranmalıdır:

- “Şimdilik böyle” diye bırakılmış kodlar,
- yarım kalmış migration ve refactor’lar,
- eski ve yeni yaklaşımın aynı anda yaşadığı bölgeler,
- geçici diye eklenmiş ama kalıcı olmuş adapter ve bridge’ler,
- TODO ve FIXME kılığına girmiş mimari borçlar.

6.8 Dördüncü Katmanın Sonucu

Bu turun sonunda şu sorulara cevap verilmelidir:

- Bu sistemin bugün bu halde olmasının en olası nedenleri nelerdir?
- En yüksek bakım yükü taşıyan alanlar hangileridir?
- Hangi modüller tarihsel ya da organizasyonel bagaj taşımaktadır?
- Koddan değil, bağlamdan kaynaklanan en büyük kısıtlar nelerdir?
- Teknik liderin refactor kararı verirken bilmesi gereken bağlamsal uyarılar nelerdir?

Bu katman sana **neden o yolun kapandığını** anlatır.

7 Bu Yapının Neden Güçlü Olduğu

Bu dört katman birlikte kullanıldığında Claude sadece dosya tarayan bir araç olmaktan çıkar; mimari, risk, öncelik ve bağlam arasında ilişki kuran bir analiz motoruna dönüşür.

Birinci tur olmadan sistemin omurgası görülmez. İkinci tur olmadan gerçek riskler seçilemez. Üçüncü tur olmadan analiz aksiyona dönüşmez. Dördüncü tur olmadan da yanlış önceliklendirme riski doğar.

Bu yüzden bu çerçeve tek bir checklist değil, ardışık karar katmanlarıdır.

8 Claude’a K r  ekilde “Oku” Demek Yerine Nasıl  er eve Verilmeli

Claude’a yalnızca “repo’yu oku” demek verimsizdir. Bunun yerine ona  u t r bir y nlendirme verilmelidir:

 nce mimariyi  ıkar. Sonra ana feature akı larını  ıkar. Sonra riskli yerleri ve coupling noktalarını bul. Sonra de i im maliyeti y ksek alanları ayıkla. En son hangi refactor’un ne zaman yapılması gerekti ini s yle. Ve m mk nse tarihsel, operasyonel ve business ba lamı da hesaba kat.

Bu y nlendirme sayesinde Claude hem dosyaları okur, hem de ne t r bir zihinsel model kurması gerekti ini anlar.

9 CLAUDE.md  retme Mantı ı

Repodan otomatik bir CLAUDE.md  ıkarttırılacaksa burada da k r taklit yapılmamalıdır. Repo i inde tekrar eden her pattern do ru de ildir. Bazı tekrarlar takım standardı olabilir; bazıları ise legacy, yarım refactor, lokal tercih ya da ge ici c z md r. Bu y zden  nce  u sınıflandırma yapılmalıdır:

9.1 Kesin G r nen Kurallar

Bunlar repo genelinde tutarlı  ekilde g r len, tooling, config ya da test ile desteklenen, y ksek g venle kural h line getirilebilecek pattern’lerdir.

9.2 Muhtemel Ama Belirsiz Kurallar

Bunlar birden fazla yerde g r len ama evrensel olmayan, bazı feature’larda ge erli olup bazılarında bozulmu  g r nen, takım standardı mı yoksa lokal tercih mi oldu u net olmayan davranı lardır.

9.3  eli kili veya G venilmez Alanlar

Bunlar repo i inde birbiriyle  eli en pattern’ler, legacy izleri, ge i  s reci kalıntıları ve kural diye yazılması tehlikeli alanlardır.

9.4 Do ru  retim Sırası

Bu nedenle sa lıklı akı    yledir:

1.  nce repo taranır,
2. sonra mimari ve organizasyon  zeti  ıkarılır,
3. ardından g venilir kural adayları ile   pheli alanlar ayrı tırılır,

4. sonra sadece yüksek güvenli kurallarla `CLAUDE.md` taslağı yazılır,
5. en son insan review gerektiren açık sorular ayrıca listelenir.

9.5 CLAUDE.md İçeriğinde Neler Olmalı

Sağlam bir taslak şu bölümleri içerebilir:

- Project overview
- Architecture summary
- Source of truth and data flow rules
- Frontend conventions
- Backend conventions
- State management conventions
- API, DTO ve model conventions
- Testing expectations
- File ve folder organization rules
- Naming conventions
- Error handling ve logging rules
- Refactoring guardrails
- What to avoid
- Safe workflow for making changes

Ayrıca ayrıca şu başlık da eklenmelidir:

Open questions before finalizing CLAUDE.md

Bu bölüm, insan review gerektiren belirsizlikleri, takım kararı gerektiren çelişkileri ve kural hâline getirilmemesi gereken ama fark edilen legacy alanları toplamalıdır.

10 Bu Çerçevenin Yeterli Olduğu ve Yetmediği Yerler

Bu yapı onboarding ve mimari keşif için oldukça güçlüdür. Yeni bir geliştiriciyi projeye sokmak, kritik dosyaları bulmak, ana akışları görmek ve teknik risk haritası çıkarmak için yeterlidir.

Ancak tek başına mucize değildir. Çünkü koddan şu şeyler her zaman tam çıkmaz:

- neden bu karar verilmişti,
- geçmişte ne patlamıştı,
- business için en kritik alan hangisi,
- takım en çok nerede zorlanıyor,
- support yükünü en çok hangi feature üretiyor.

Bu yüzden repo yanında mümkünse şu ek bağlamlar da verilmelidir:

- eski PR açıklamaları,
- bug kayıtları,
- incident ve postmortem notları,
- release note'lar,
- support şikayetleri,
- migration notları,
- TODO ve FIXME yoğun alanlar,
- setup sırasında ekipçe yaşanan sıkıntılar.

11 En Net Sonuç

Claude'a proje okutmanın en doğru çerçevesi, tek seferlik dosya taraması değil; katmanlı ve amaç odaklı bir analiz düzenidir.

Birinci katman sistemi tanıtır. İkinci katman kırılganlığı çıkarır. Üçüncü katman aksiyon ve öncelik sırası üretir. Dördüncü katman ise kodun arkasındaki gerçek tarihi, baskıyı ve sahadaki acıyı anlamaya çalışır.

Dolayısıyla temel hüküm şudur:

Claude'a dosya okutma. Claude'a sistem düşündür. Önce haritayı çizdir. Sonra mayın tarlasını buldur. Sonra hangi yolu açacağını seçtir. En sonunda neden o yolun kapandığını sorgulat.

İşte sağlam, profesyonel ve gerçekten kullanılabilir proje analizi yaklaşımı budur.

12 Test Rig Kavramı ve Alt Bileşenleri

12.1 Genel Tanım

Test rig, bir yazılım sistemini, bir akışı ya da belirli bir özelliği kontrollü biçimde sınamak için kurulan **test düzeneği**, **test altyapısı** veya **yardımcı test kurulumu** anlamına gelir. Bu nedenle “test rig” tek bir şeye karşılık gelmez; aksine, testleri mümkün kılan veya kolaylaştıran birçok alt parçanın birleşiminden oluşur.

Bu parçaların isimleri sektörde her zaman tek bir standart altında toplanmaz. Ancak pratikte bir test rig çoğunlukla aşağıdaki bileşen türlerinden meydana gelir.

12.2 1. Test Data Katmanı

Bu katman, sistemin anlamlı bir duruma sokulabilmesi için kullanılan verileri içerir.

- **Mock data:** Gerçeği taklit eden örnek veri
- **Sample data:** Gösterim veya geliştirme amaçlı örnek veri
- **Seed data:** Sistemi başlangıç durumuna getirmek için yüklenen veri
- **Fixture data:** Test senaryolarında tekrar tekrar kullanılan sabit veri
- **Synthetic data:** Yapay olarak üretilmiş veri
- **Golden data / reference data:** Doğruluğu referans kabul edilen veri
- **Dummy records:** Basit doldurma amaçlı kayıtlar
- **Preloaded scenarios:** Önceden hazırlanmış örnek senaryo verileri

12.3 2. Test Doubles

Gerçek bağımlılıkların yerine geçen ve test sırasında kullanılan yapılar bu gruba girer.

- **Mock:** Beklenen çağrılar ve davranışları doğrulayan test nesnesi
- **Stub:** Belirli girdilere sabit cevap dönen sahte bileşen
- **Fake:** Gerçeğe benzeyen ama daha basit çalışan uygulama
- **Spy:** Hangi çağrının yapıldığını kaydeden yardımcı nesne
- **Dummy object:** Sadece yer tutmak için kullanılan nesne
- **Service virtualization:** Gerçek dış servislerin sanallaştırılmış hali
- **Emulator:** Gerçek sistemin davranışını daha yakından taklit eden yapı
- **Simulator:** Belirli davranışları modelleyen test bileşeni

12.4 3. Environment / Runtime Setup

Testin çalıştığı ortamı ifade eder.

- **Local test environment:** Yerel makinede çalışan test ortamı
- **Sandbox:** İzole deneme ortamı
- **Staging environment:** Canlıya yakın ön üretim ortamı
- **Ephemeral environment:** Geçici oluşturulan kısa ömürlü test ortamı
- **Containerized test environment:** Docker veya benzeri kapsüllenmiş ortam
- **In-memory environment:** Bellek üzerinde çalışan hafif test ortamı
- **Dedicated QA environment:** Kalite güvencesi için ayrılmış ortam
- **Preview environment:** Özelliğe özel önizleme ortamı

12.5 4. Infrastructure Pieces

Testi teknik olarak ayağa kaldıran altyapı bileşenleridir.

- **Test database**
- **Seeded database**
- **Message queue test instance**
- **Cache test instance**
- **Object storage test bucket**
- **Fake SMTP server**
- **Local file storage**
- **Test auth provider**
- **Feature-flag configuration**
- **Config overrides**

12.6 5. Execution Helpers

Testlerin başlatılmasını, hızlandırılmasını veya tekrarlanmasını kolaylaştıran araçlardır.

- **Test runner**
- **Test harness**
- **Bootstrap script**
- **Setup script**
- **Teardown script**
- **CLI helper**
- **One-click dev action**
- **Populate button / autofill button**
- **Reset script**
- **Replay script**

12.7 6. Observability ve Debugging Yardımcıları

Test sırasında ne olduğunu görünür kılan parçalar bu gruptadır.

- **Verbose logs**
- **Debug panel**
- **Event viewer**
- **Request inspector**
- **Response inspector**
- **Trace viewer**
- **Metrics dashboard**
- **Screenshot/video capture**
- **State inspector**

12.8 7. Control Mechanisms

Test koşullarını bilinçli biçimde değiştirmeye yarayan mekanizmalardır.

- Feature toggles
- Dev-only switches
- Scenario selector
- Latency injection
- Fault injection
- Network throttling
- Clock/time control
- Random seed control
- Permission/role switcher

12.9 8. Verification Pieces

Sistemin doğru çalışıp çalışmadığını sıyanan doğrulama bileşenleridir.

- Assertions
- Expected outputs
- Snapshot baselines
- Golden file comparison
- Contract checks
- Schema validation
- Response diffing
- DOM/state verification
- Data integrity checks

12.10 9. Cleanup ve Reset Mekanizmaları

Test sonrası ortamı temiz ve yeniden kullanılabilir duruma getiren parçalardır.

- Database reset
- State reset
- Cache clear

- Queue purge
- Temp file cleanup
- Test user cleanup
- Environment rollback
- Idempotent reset action

12.11 10. Scenario Packs

Önceden tanımlanmış test akışları veya senaryo kümeleri bu başlık altında düşünülebilir.

- Happy path scenario
- Edge case scenario
- Failure scenario
- Partial data scenario
- Large data scenario
- Permission-restricted scenario
- Offline / degraded mode scenario
- Migration scenario

12.12 Yakın Kavramlar

Aşağıdaki kavramlar çoğu zaman test rig ile karıştırılır. Bunlar bazen test rig'in parçasıdır, bazen ise ayrı kavramlar olarak değerlendirilir.

- Test harness
- Fixture
- Seed script
- Sandbox
- Staging
- QA environment
- Integration test suite
- Smoke test setup

12.13 Sonuç

Dolayısıyla “test rig” denildiğinde, bundan tek bir test türü değil; testleri mümkün kılan, hızlandıran, gözlemlenebilir hale getiren ve tekrar üretilebilir yapan **çok parçalı bir yardımcı sistem bütünü** anlaşılmalıdır.

Senin örneğindeki sarı buton için en uygun teknik ifade şu şekilde yazılabilir:

This button is a dev-only mock data seeding helper that is part of our test rig.

Sonuç

Bu belgede ele alınan iki konu, ilk bakışta birbirinden bağımsız görünse de aynı temel mühendislik ilkesine dayanmaktadır: karmaşıklığı yönetmenin yolu, onu doğru katmanlara ve doğru kavramlara ayırmaktan geçer.

Claude’a proje okutma çerçevesi açısından varılan sonuç nettir. Bir kod tabanını yapay zekâya analiz ettirmenin doğru yolu, dosya boyutuna göre sıralanmış mekanik bir tarama değil; her turu farklı bir soruya cevap veren katmanlı bir analiz düzenidir. Birinci katman sistemin haritasını çıkarır ve mimari omurgayı görünür kılar. İkinci katman kırılganlıkları, coupling noktalarını ve değişim maliyeti yüksek bölgeleri ayıklayarak mayın tarlasını gösterir. Üçüncü katman bu bulguları önceliklendirilmiş bir aksiyon planına dönüştürür. Dördüncü katman ise kodun neden bu hâlde olduğunu — tarihsel kararları, geçmiş arızaları, takımın gerçek acılarını ve iş kritikliğini — hesaba katarak analizi bağlama oturur. Bu katmanlardan herhangi biri eksik kaldığında analiz ya yüzeysel kalır, ya aksiyona dönüşmez, ya da teknik olarak doğru ama iş açısından yanlış öncelikler üretir. Aynı disiplin **CLAUDE.md** üretiminde de geçerlidir: repoda görülen her pattern kural değildir; güvenilir kurallar, belirsiz adaylar ve çelişkili alanlar birbirinden ayrıştırılmadan yazılan bir kural dosyası, yardımdan çok zarar üretir.

Test rig kavramı açısından ise varılan sonuç şudur: test rig tek bir araç ya da tek bir test türü değil, testleri mümkün kılan, hızlandıran, gözlemlenebilir hâle getiren ve tekrar üretilebilir yapan çok parçalı bir yardımcı sistem bütünüdür. Veri katmanından test doubles yapılarına, ortam kurulumundan kontrol mekanizmalarına, doğrulama bileşenlerinden temizlik rutinlerine kadar uzanan bu taksonomi, hem bir sistemin test edilebilirliğini değerlendirmek hem de eksik parçaları tespit etmek için kullanılabilir bir kontrol çerçevesi sunar.

Sonuç olarak, her iki bölüm de aynı yargıyı destekler: iyi mühendislik, araçları körü körüne kullanmak değil, onlara doğru zihinsel modeli ve doğru çalışma düzenini vermektir. Yapay zekâya sistem düşündürmek de, sağlam bir test altyapısı kurmak da ancak bu katmanlı ve amaç odaklı yaklaşımla mümkündür.